

PKCERT AI & Software Development Internship

Task 27 – NLP Basics: Tokenization, Word Embeddings (Word2Vec/GloVe), and Introduction to Hugging Face Transformers

Objective: Build a rigorous conceptual and practical foundation in core Natural Language Processing preprocessing and representation techniques. Interns will formally define and implement text tokenization strategies, derive and reason precisely about distributed word embedding methods (Word2Vec and GloVe), and gain hands-on familiarity with the Hugging Face transformers ecosystem for loading tokenizers, pretrained models, and extracting contextual embeddings. This task bridges the classical NLP pipeline (Tasks 22 and 25) with the applied Transformer analysis introduced in Task 26, establishing the representational groundwork required for downstream fine-tuning and applied model work.

Total Marks: 100

Part A – Tokenization Foundations (20 Marks)

- Formally define the tokenization problem in NLP and distinguish between word-level, character-level, and subword-level tokenization schemes, explaining the trade-offs each makes with respect to vocabulary size, out-of-vocabulary (OOV) handling, and sequence length.
- Explain the motivation for subword tokenization algorithms and derive, step by step, how Byte-Pair Encoding (BPE) constructs a merge-based vocabulary from a training corpus, including the iterative pair-frequency counting and merge procedure.
- Compare BPE against WordPiece and SentencePiece/Unigram tokenization, explaining the core algorithmic difference in how each selects merges or subword units (frequency-based vs. likelihood-based), and identify which tokenizer is used by which family of pretrained models (e.g., GPT-style BPE, BERT WordPiece, T5/ALBERT SentencePiece).
- Implement a from-scratch, simplified BPE tokenizer in Python on a small custom text corpus (no external tokenizer libraries for this sub-task), demonstrating vocabulary construction and encoding/decoding of a sample sentence, and report the resulting vocabulary and merge sequence.
- Discuss the practical preprocessing considerations that precede tokenization (lowercasing, Unicode normalization, punctuation handling, special tokens such as [CLS], [SEP], [PAD], [UNK]) and their downstream effects on model input formatting.

Part B – Word Embeddings: Word2Vec & GloVe (25 Marks)

- Explain the distributional hypothesis underlying word embeddings and contrast sparse representations (one-hot, bag-of-words, TF-IDF) with dense distributed representations, motivating the shift toward learned embeddings.
- Formally derive the Word2Vec Skip-gram objective, including the softmax formulation over the vocabulary, and explain precisely why negative sampling is introduced as a computationally tractable approximation to the full softmax; state the negative sampling loss function and interpret each term.
- Contrast the Skip-gram and Continuous Bag-of-Words (CBOW) architectures, explaining the difference in prediction direction (context→target vs. target→context) and the practical implications for training data efficiency and embedding quality on rare vs. frequent words.
- Explain the GloVe (Global Vectors) formulation: derive the co-occurrence matrix construction, state the weighted least-squares objective function, and explain conceptually why GloVe combines global co-occurrence statistics with the local context-window intuition of Word2Vec.

- Train a Word2Vec model (using an established library such as *gensim*) on a modest text corpus, report the resulting nearest-neighbour words for at least three chosen query words, and evaluate a small set of word analogies (e.g., king – man + woman ≈ queen) using vector arithmetic on the trained embeddings.
- Load pretrained GloVe vectors, repeat the nearest-neighbour and analogy evaluation, and compare qualitatively against the self-trained Word2Vec embeddings, discussing differences attributable to training corpus size and objective formulation.
- Visualize a selected subset of word embeddings (Word2Vec and/or GloVe) in two dimensions using a dimensionality-reduction technique (PCA or t-SNE), and interpret any observed semantic or syntactic clustering.

Part C – Applied Hugging Face Transformers Analysis (35 Marks)

- Using the Hugging Face *transformers* and *tokenizers* libraries, load a pretrained tokenizer (e.g., from BERT, RoBERTa, or DistilBERT) and demonstrate its behaviour on a sample sentence: show the resulting token sequence, input IDs, attention mask, and any special tokens inserted, contrasting this output directly with the from-scratch BPE tokenizer implemented in Part A.
- Load a corresponding pretrained model (e.g., via AutoModel) and extract static, non-contextual-layer embeddings from the model's input embedding matrix for a small set of words, comparing these against the Word2Vec/GloVe embeddings from Part B for the same words using cosine similarity.
- Extract contextual embeddings (final or intermediate hidden-layer outputs) for the same words used above, appearing in at least two different sentence contexts, and demonstrate quantitatively (via cosine similarity) how the contextual embedding for a polysemous word changes across contexts, contrasting this against the fixed, context-independent nature of Word2Vec/GloVe vectors.
- Explain, with reference to the tokenization and embedding pipeline used, how subword tokenization interacts with embedding lookup for out-of-vocabulary or rare words, and how this differs from the OOV handling (or lack thereof) in classical Word2Vec/GloVe models.
- Summarize, at a pipeline level, how the tokenizer and embedding layer examined in this task feed into the attention mechanism and Transformer block architecture studied in Task 26, explicitly connecting the input representation stage to the downstream self-attention computation.

Part D – Analysis & Documentation (20 Marks)

- Summarize the full representational pipeline covered in this task: tokenization strategy, sparse vs. dense representations, Word2Vec and GloVe formulations, and Hugging Face tokenizer/embedding extraction, explaining the reasoning for each stage's role in the overall NLP pipeline.
- Summarize the quantitative findings from Part B (nearest-neighbour and analogy results) and Part C (static vs. contextual embedding cosine-similarity comparisons), presenting the key results in tabular form.
- Discuss at least two non-trivial conceptual or implementation challenges encountered while implementing the from-scratch BPE tokenizer, training the Word2Vec model, or extracting Hugging Face embeddings, and explain precisely how each was resolved.
- Reflect critically on the limitations of static word embeddings (inability to capture polysemy, fixed vocabulary, lack of context-sensitivity) relative to contextual Transformer-based embeddings, and identify at least one scenario where a lightweight static embedding approach remains preferable to a full Transformer pipeline (e.g., resource-constrained deployment, interpretability needs).

Assessment Breakdown

Assessment Area	Marks
-----------------	-------

Tokenization Foundations	20
Word Embeddings: Word2Vec & GloVe	25
Applied Hugging Face Transformers Analysis	35
Analysis & Documentation	20
Total	100

Note: Part A requires a from-scratch, simplified BPE implementation for pedagogical purposes only; Parts B and C must use established libraries (gensim, pretrained GloVe vectors, Hugging Face transformers/tokenizers) rather than reimplementing Word2Vec, GloVe, or any Transformer component from scratch. All derivations must be shown mathematically in the document; code is required only where explicitly specified in Parts A, B, and C.